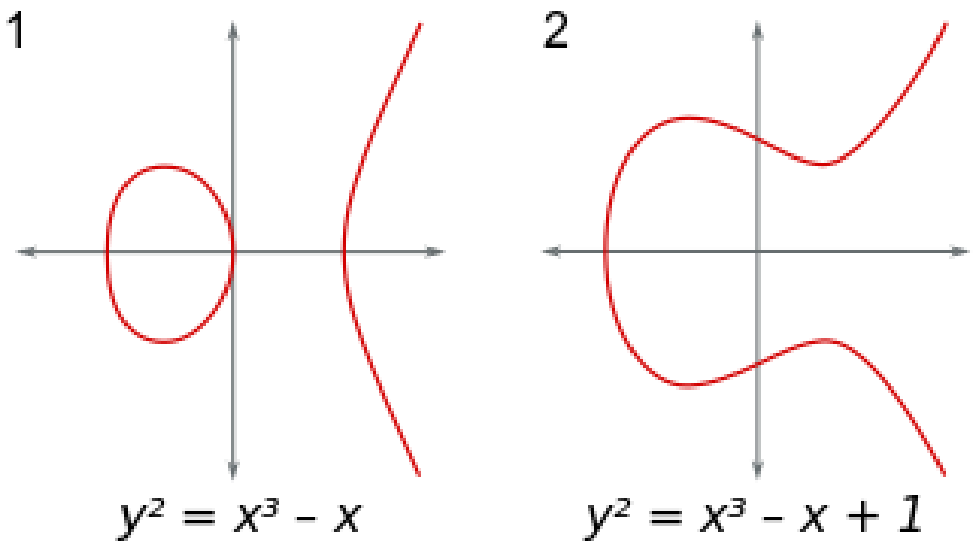
EC and ED

Paul Joo-Hyun Kim (pauljoohyunkim@gmail.com)

January 6, 2026

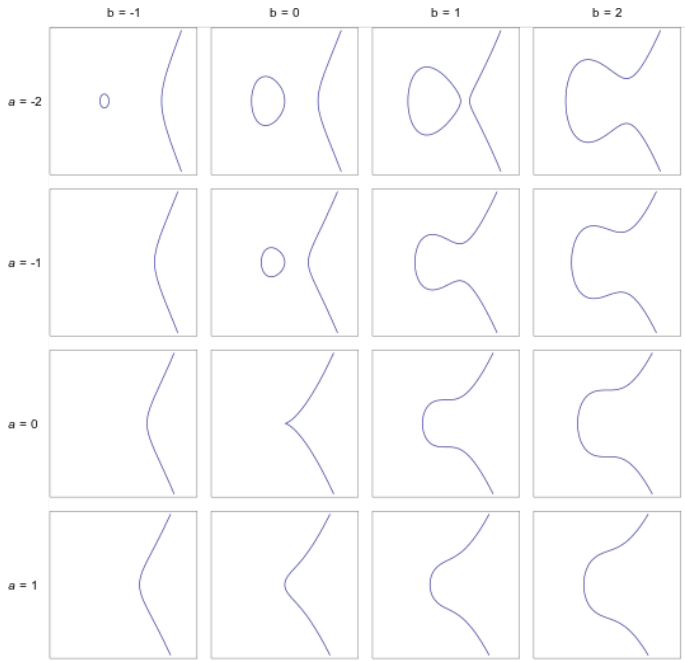
Elliptic Curve



Elliptic Curve

$y^2 = x^3 + ax + b$ over $a, b \in K$ where K is a field.

Elliptic Curve



Condition

Need to satisfy $4a^3 + 27b^2 \neq 0$ for no cusps or self intersection

Understanding Group and Field

Analogy

Real Number(Field)	RSA(Field)	Elliptic Curve(Group)
Number	Remainder	Point
Addition	Mod Add.	EC Addition
Multiplication	Mod Mult.	N/A

Group

A set with a binary operation that is associative, has an identity, with all element has an inverse, and is closed.

- Set of invertible matrices with multiplication.
- Set of integers with addition.

Field

A group with addition defined as group operation, and multiplication defined as group operation (without zero).

- Set of rational numbers with multiplication.

Group and Field

"Class" Pseudocode

```
1  class GroupElement {
2      GroupElement(RawValue val=Identity);
3      GroupElement operator+ (GroupElement h);
4      GroupElement add_inv();
5
6      RawValue val;
7  };
8
9  class FieldElement : public GroupElement {
10     FieldElement(RawValue val=Add_Identity);
11     FieldElement(RawValue val=Mul_Identity);
12     FieldElement operator* (FieldElement h);
13     FieldElement mul_inv();      // Non-"zero" only.
14
15     RawValue val;
16 }
```

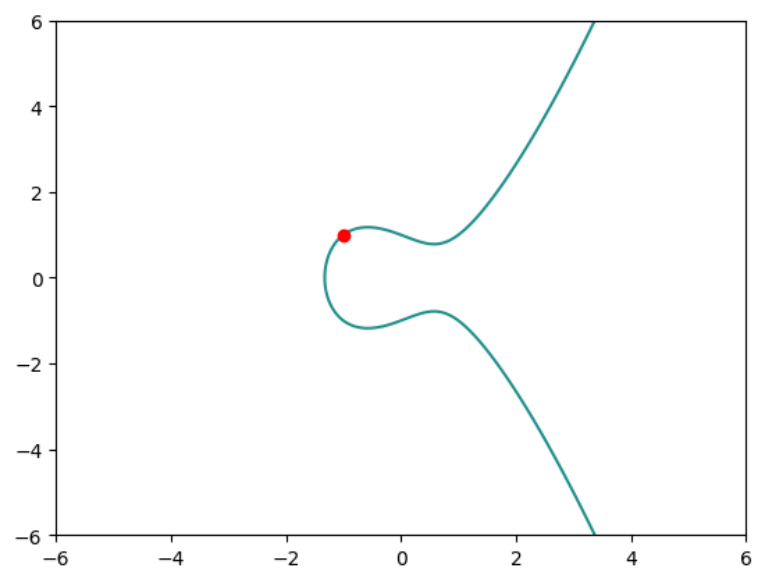
Elliptic Curve Group

Elliptic Curve Group

For elliptic curve $y^2 = x^3 + ax + b$ on field K, let points form a group E (with infinity being the identity).

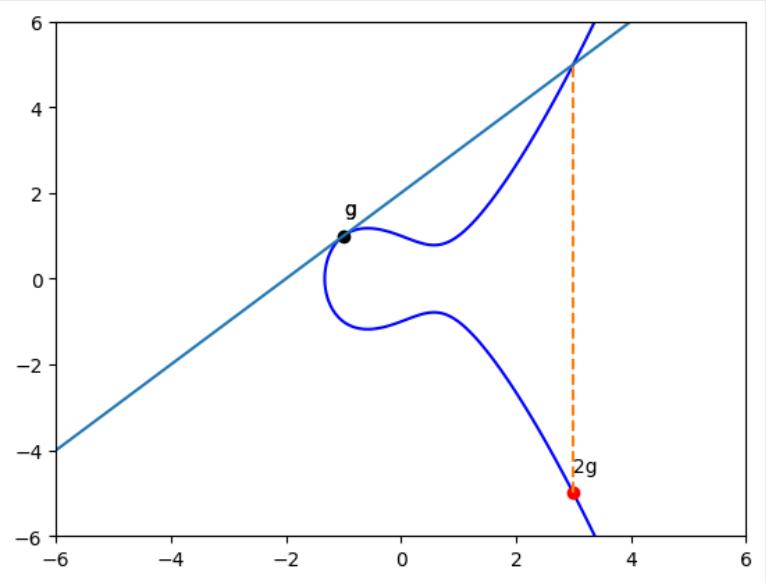
Point Addition

$G \in E$ is the red point.



Point Addition (Cont'd)

$2G = G + G$ (Tangent)



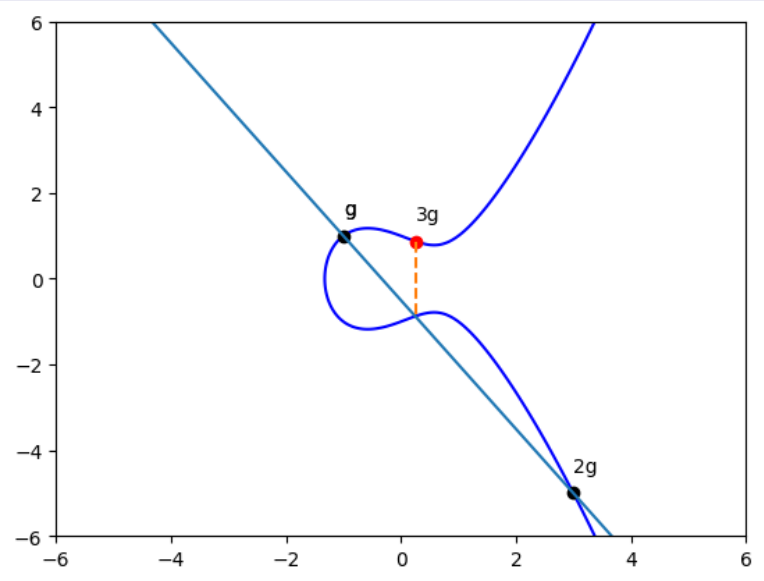
$$s = \frac{3x_G^2 + a}{2y_G}$$

$$x_{2G} = s^2 - 2x_G$$

$$y_{2G} = s(x_G - x_{2G}) - y_G$$

Point Addition (Cont'd)

$3G = G + 2G$



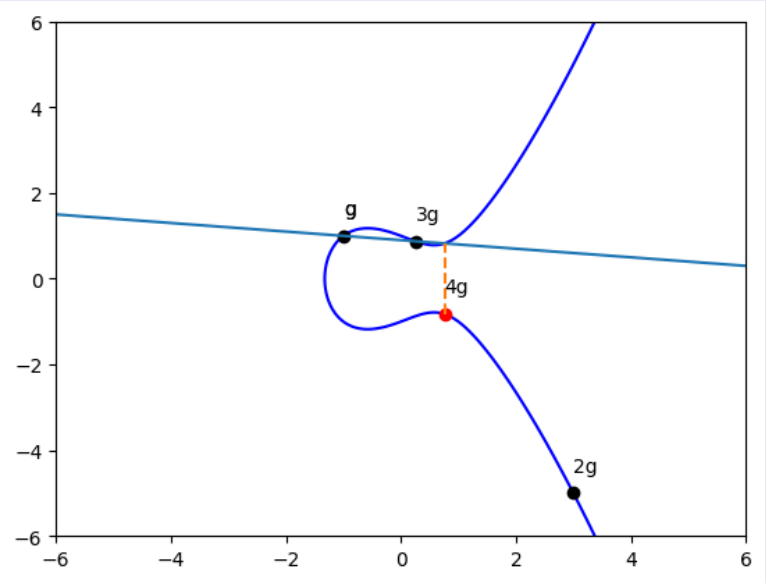
$$s = \frac{y_P - y_Q}{x_P - x_Q}$$

$$x_{P+Q} = s^2 - x_P - x_Q$$

$$y_{P+Q} = s(x_P - x_{P+Q}) - y_P$$

Point Addition (Cont'd)

$$4G = G + 3G$$



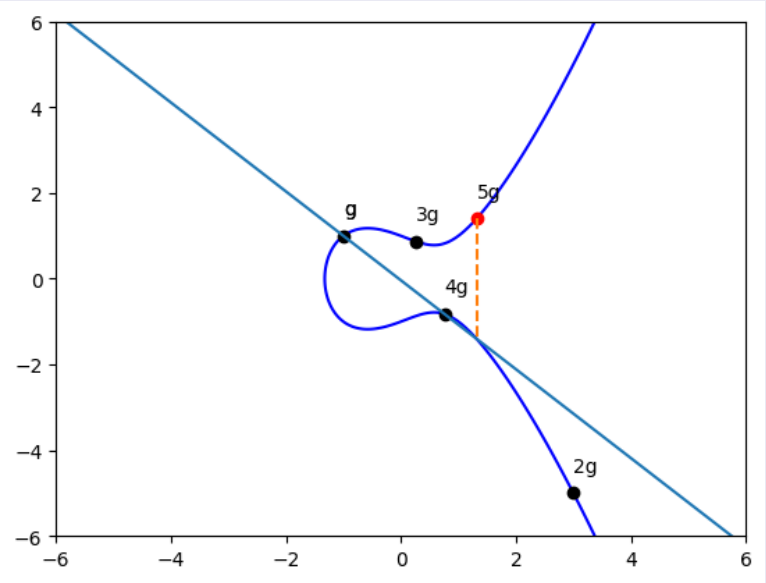
$$s = \frac{y_P - y_Q}{x_P - x_Q}$$

$$x_{P+Q} = s^2 - x_P - x_Q$$

$$y_{P+Q} = y_P - s(x_P - x_{P+Q})$$

Point Addition (Cont'd)

$$5G = G + 4G$$



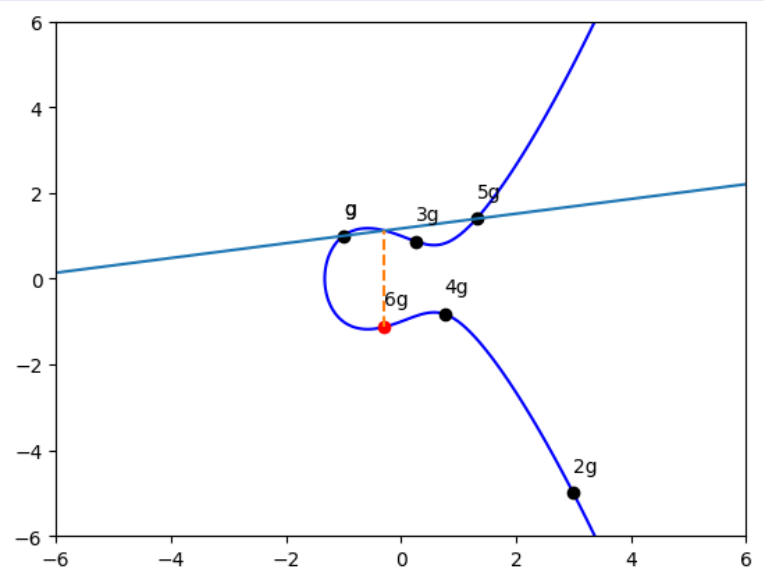
$$s = \frac{y_P - y_Q}{x_P - x_Q}$$

$$x_{P+Q} = s^2 - x_P - x_Q$$

$$y_{P+Q} = y_P - s(x_P - x_{P+Q})$$

Point Addition (Cont'd)

$$6G = G + 5G$$



$$s = \frac{y_P - y_Q}{x_P - x_Q}$$

$$x_{P+Q} = s^2 - x_P - x_Q$$

$$y_{P+Q} = y_P - s(x_P - x_{P+Q})$$

Elliptic Curve Group (Cryptography)

Operation

The formulae stay the same, but now on modulo p.

Formulae (Modulo)

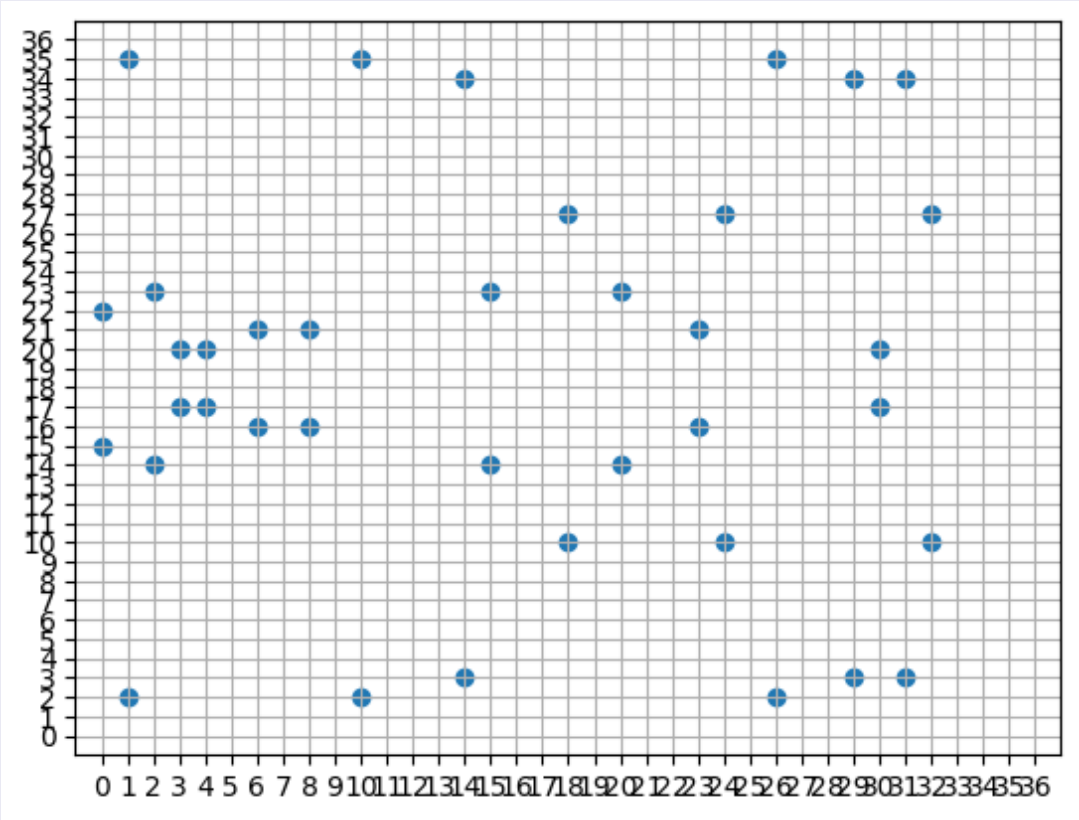
Doubling

$$s = \left(3x_G^2 + a\right)\left(2y_G\right)^{-1}$$
$$x_{2G} = s^2 - 2x_G$$
$$y_{2G} = y_G - s(x_G - x_{2G})$$

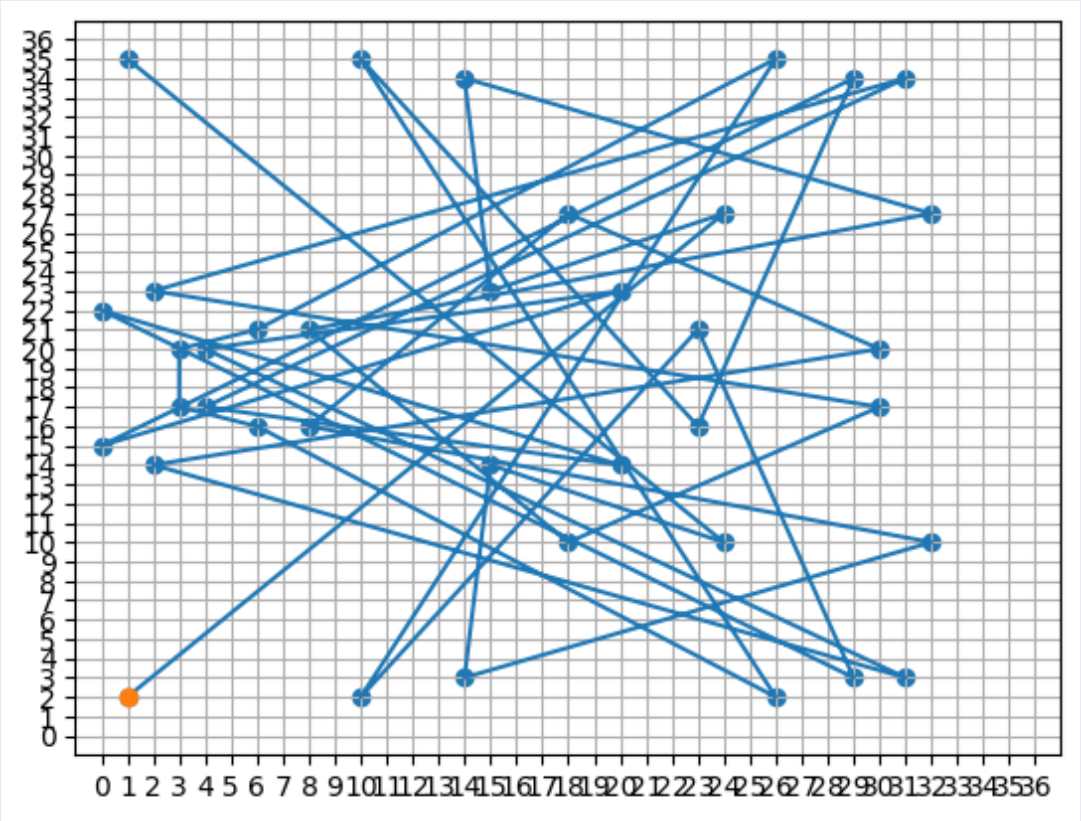
Addition

$$s = (y_P - y_Q)(x_P - x_Q)^{-1}$$
$$x_{P+Q} = s^2 - x_P - x_Q$$
$$y_{P+Q} = y_P - s(x_P - x_{P+Q})$$

Modulo 37, $a = 0, b = 3$



Isomorphism: Relation to Modular Arithmetics



Isomorphic to $\mathbb{F}_{39} = \{0, 1, \cdots, 38\}$

Elliptic Curve Cryptography

NIST-P256

$$y^2 = x^3 + ax + b \mod p$$

- p
  - 0xffffffff00000001000000000000000000000000ffffffffffffffffffffffff
- a
  - 0xffffffff00000001000000000000000000000000fffffffffffffffffffffffc
- b
  - 0x5ac635d8aa3a93e7b3ebbd55769886b
    c651d06b0cc53b0f63bce3c3e27d2604b

NIST-P256 (Cont'd)

- $G = (G_x, G_y)$ (Public)
  - 0x6b17d1f2e12c4247f8bce6e563a440f27 7037d812deb33a0f4a13945d898c296
  - 0x4fe342e2fe1a7f9b8ee7eb4a7c0f9e162 bce33576b315ececbb6406837bf51f5

Key Pair

Let d be CSRN. d is **private key**. Compute dG. dG is **public key**. In NIST-P256, d, $(dG)_x$, and $(dG)_y$ are all 32-bytes.

ECDH

Shared Secret Generation

- Alice
  - d_A
  - $d_A G$
- Bob
  - d_B
  - $d_B G$

Published: $d_A G$, $d_B G$, G

- Alice generates shared secret by $d_A \times d_B G$
- Bob generates shared secret by $d_B \times d_A G$.

Optimization of ECC

Remark

To compute $P(x_P, y_P) + Q(x_Q, y_Q)$, one could use

$$s = (y_P - y_Q)\underbrace{(x_P - x_Q)^{-1}}_{\text{Expensive}}$$

$$x_{P+Q} = s^2 - x_P - x_Q$$

$$y_{P+Q} = y_P - s(x_P - x_{P+Q})$$

But this is expensive.

Optimization of ECC

Jacobian Coordinate

Let $x = \frac{X}{Z^2}$ and $y = \frac{Y}{Z^3}$, then use optimized formulae.

1998 Cohen/Miyaji/Ono, formula (5):

```
K<a,b,X1,Y1,X2,Y2>:=FieldOfFractions(PolynomialRing(Rationals(),6));
R<Z1,Z2>:=PolynomialRing(K,2);
S:=quo<R|Y1^2-X1^3-a*X1*Z1^4-b*Z1^6,Y2^2-X2^3-a*X2*Z2^4-b*Z2^6>;
x1:=X1/Z1^2; y1:=Y1/Z1^3;
S!(y1^2-x1^3-a*x1-b);
x2:=X2/Z2^2; y2:=Y2/Z2^3;
S!(y2^2-x2^3-a*x2-b);
lambda:=(y2-y1)/(x2-x1);
x3:=lambda^2-x1-x2; y3:=lambda*(x1-x3)-y1;
S!(y3^2-x3^3-a*x3-b);
// here are the formulas:
U1:=X1*Z2^2;
U2:=X2*Z1^2;
S1:=Y1*Z2^3;
S2:=Y2*Z1^3;
H:=U2-U1;
r:=S2-S1;
X3:=-H^3-2*U1*H^2+r^2;
Y3:=-S1*H^3+r*(U1*H^2-X3);
Z3:=Z1*Z2*H;
S!(x3-X3/Z3^2); S!(y3-Y3/Z3^3);
```

Eliminating common subexpressions from 1998 Cohen/Miyaji/Ono, 12M + 4S + 6add + 1times2:

```
K<a,b,X1,Y1,X2,Y2>:=FieldOfFractions(PolynomialRing(Rationals(),6));
R<Z1,Z2>:=PolynomialRing(K,2);
S:=quo<R|Y1^2-X1^3-a*X1*Z1^4-b*Z1^6,Y2^2-X2^3-a*X2*Z2^4-b*Z2^6>;
x1:=X1/Z1^2; y1:=Y1/Z1^3;
S!(y1^2-x1^3-a*x1-b);
x2:=X2/Z2^2; y2:=Y2/Z2^3;
S!(y2^2-x2^3-a*x2-b);
lambda:=(y2-y1)/(x2-x1);
x3:=lambda^2-x1-x2; y3:=lambda*(x1-x3)-y1;
// here are the formulas:
Z1Z1:=Z1^2;
Z2Z2:=Z2^2;
U1:=X1*Z2Z2;
U2:=X2*Z1Z1;
S1:=Y1*Z2*Z2Z2;
S2:=Y2*Z1*Z1Z1;
H:=U2-U1;
HH:=H^2;
HHH:=H*HH;
r:=S2-S1;
```

Public Key Compression for EC

SEC-1 Format

```
{
    EVP_PKEY* pkey_raw { nullptr };
    OSSL_PARAM ctx_params[2] {
        OSSL_PARAM_construct_utf8_string(OSSL_PKEY_PARAM_GROUP_NAME, "P-256", 0),
        OSSL_PARAM_construct_end()
    };

    // Generate EC P256 key pair using OpenSSL
    ctx = decltype(ctx)(EVP_PKEY_CTX_new_from_name(nullptr, "EC", nullptr), EVP_PKEY_CTX_free);
    EVP_FAIL_THROW(EVP_PKEY_keygen_init(ctx.get()));
    EVP_PKEY_CTX_set_params(ctx.get(), ctx_params);

    EVP_PKEY_generate(ctx.get(), &pkey_raw);
    pkey = decltype(pkey)(pkey_raw, EVP_PKEY_free);

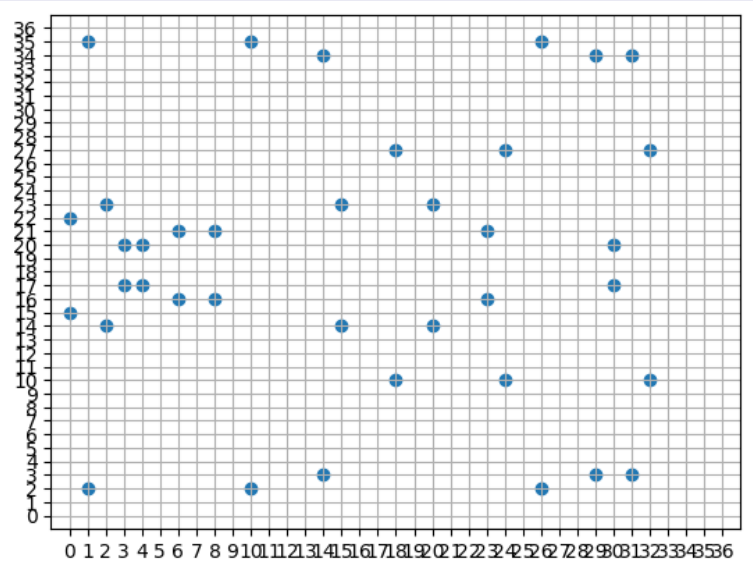
    // Extract the keys into bytes.
    // Pubkey
    OSSL_PARAM pubkey_params[2] {
        OSSL_PARAM_construct_utf8_string(OSSL_PKEY_PARAM_EC_POINT_CONVERSION_FORMAT, OSSL_PKEY_EC_POINT_CONVERSION_FORMAT_COMPRESSED, 0),
        OSSL_PARAM_construct_end()
    };
    EVP_FAIL_THROW(EVP_PKEY_set_params(pkey.get(), pubkey_params));
    EVP_FAIL_THROW(EVP_PKEY_get_octet_string_param(pkey.get(), OSSL_PKEY_PARAM_PUB_KEY, nullptr, 0, &keylen));
    pubkey.resize(keylen);
    EVP_FAIL_THROW(EVP_PKEY_get_octet_string_param(pkey.get(), OSSL_PKEY_PARAM_PUB_KEY, pubkey.data(), keylen, &keylen));

    // Privkey (at most 32-bytes for P-256)
    privkey.resize(P256_PRIVKEY_MAX_LEN);
    BIGNUM* priv_bn_raw { nullptr };
    EVP_FAIL_THROW(EVP_PKEY_get_bn_param(pkey.get(), OSSL_PKEY_PARAM_PRIV_KEY, &priv_bn_raw));
    std::unique_ptr<BIGNUM, decltype(&BN_free)> priv_bn { priv_bn_raw, BN_free };
    keylen = BN_bn2bin(priv_bn.get(), privkey.data());
    privkey.resize(keylen);
}
```

SEC-1 Format

Idea

Given x for $y^2 = x^3 + ax + b$, note that there are two y values. Instead of using 32 bytes for y, only store x, and "parity" (1 byte) to reconstruct y value.



SEC-1 Format

Encoding

Given (x, y), write:

$$0x02||x \quad \text{if } y \text{ even}$$
$$0x03||x \quad \text{if } y \text{ odd}$$
$$0x04||x||y$$

Decoding

Let $u \equiv x^3 + ax + b \mod p$. Since $p \equiv 3 \mod 4$,

1. Check if $u^{\frac{p-1}{2}} \equiv 1 \mod p$.
2. Compute $y \equiv \pm u^{\frac{p+1}{4}} \mod p$ and choose based on parity.

Twisted Edwards Curve

Problem of ECC

- For private key: 32 bytes (Good)
- For public key: 64 bytes, can be compressed to 33 bytes. (1 byte needed for a single parity bit)
- Dealing with infinity (Cumbersome)
- Parameters chosen by output of SHA-1 with a seed (Not Nothing-up-my-sleeve)
  - See Streebog() hash function, DES, etc.

Twisted Edwards Curve (for ED25519)

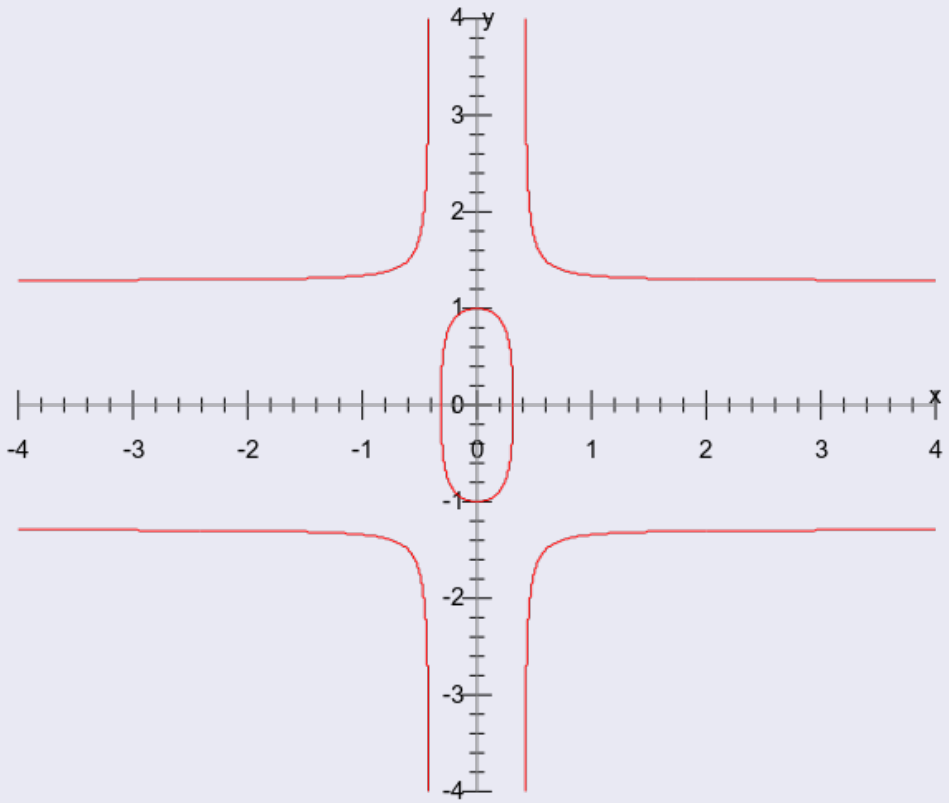
$$ax^2 + y^2 = 1 + dx^2y^2$$

where

$$(x_1, y_1) + (x_2, y_2) = \left(\frac{x_1y_2 + y_1x_2}{1 + dx_1x_2y_1y_2}, \frac{y_1y_2 - ax_1x_2}{1 - dx_1x_2y_1y_2} \right)$$

with identity being $(0, 1)$

$10x^2 + y^2 = 1 + 6x^2y^2$



ED25519

ED25519 Parameters

For $ax^2 + y^2 = 1 + dx^2y^2 \mod p$,

- p: 0x7fffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffed
  - $p = 2^{255} - 19$
- a: 0x7ffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffec
  - $a \equiv -1$
- d: 0x52036cee2b6ffe738cc740797779e898
  00700a4d4141d8ab75eb4dca135978a3
- G:
  - x: 0x216936D3CD6E53FEC0A4E231FDD6DC5C
    692CC7609525A7B2C9562D608F25D51A
  - y: 0x6666666666666666666666666666666
    6666666666666666666666666666658

Optimization

Extended twisted Edwards (ETED) Coordinate Addition

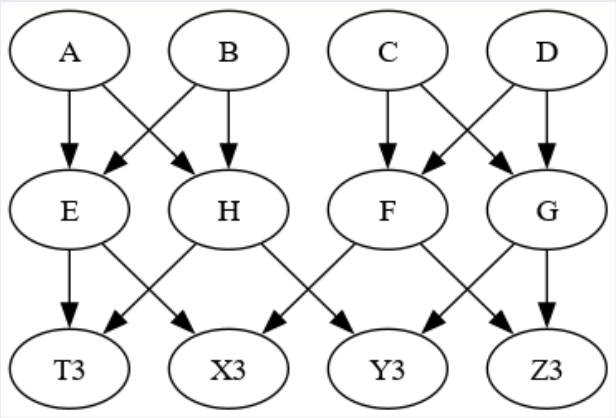
Let $x = \frac{X}{Z}$, $y = \frac{Y}{Z}$, $xy = \frac{T}{Z}$.

The "madd-2008-hwcd" addition formulas [database entry; Sage verification script; Sage ou

- Assumptions: Z2=1.
- Cost: 8M + 1*a + 1*d + 7add.
- Cost: 8M + 1*a + 6add dependent upon the first point.
- Source: 2008 Hisil–Wong–Carter–Dawson, http://eprint.iacr.org/2008/522, Section 3.1.
- Strongly unified.
- Explicit formulas:

```
A = X1*X2
B = Y1*Y2
C = T1*d*T2
D = Z1
E = (X1+Y1)*(X2+Y2)-A-B
F = D-C
G = D+C
H = B-a*A
X3 = E*F
Y3 = G*H
T3 = E*H
Z3 = F*G
```

Optimization of ETED Operation (Addition)



Exact algorithm redacted

Optimization

Extended twisted Edwards (ETED) Coordinate Double

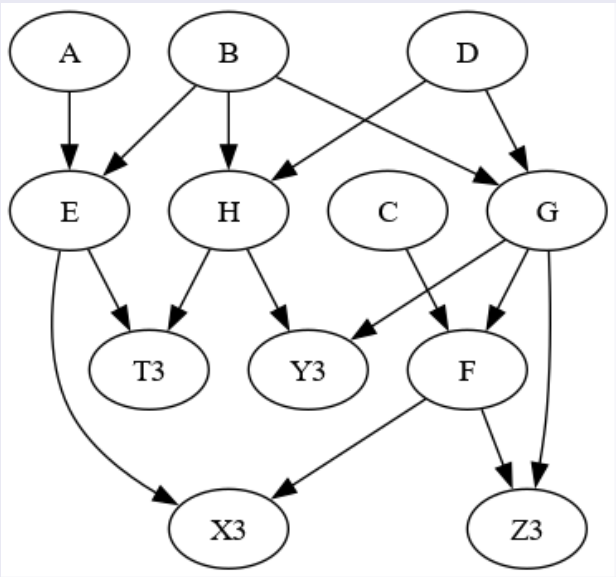
Let $x = \frac{X}{Z}$, $y = \frac{Y}{Z}$, $xy = \frac{T}{Z}$.

The "dbl-2008-hwcd" doubling formulas [database entry; Sage verification script; Sage outp

- Cost: 4M + 4S + 1*a + 6add + 1*2.
- Source: 2008 Hisil–Wong–Carter–Dawson, http://eprint.iacr.org/2008/522, Section 3.3.
- Explicit formulas:

$$
\begin{aligned}
A &= X1^2 \\
B &= Y1^2 \\
C &= 2*Z1^2 \\
D &= a*A \\
E &= (X1+Y1)^2-A-B \\
G &= D+B \\
F &= G-C \\
H &= D-B \\
X3 &= E*F \\
Y3 &= G*H \\
T3 &= E*H \\
Z3 &= F*G
\end{aligned}
$$

Optimization of ETED Operation (Double)



Exact algorithm redacted

ED Compression and Decompression

Point Compression and Decompression

Given $(x, y) \in E$ (which is 64 bytes), it can be compressed to 32 bytes. (Due to $p = 2^{255} - 19 < 2^{256}$, and the algebraic equation is quadratic with respect to y)

5.1.2. Encoding

All values are coded as octet strings, and integers are coded using little-endian convention, i.e., a 32-octet string h h[0],...h[31] represents the integer h[0] + 2^8 * h[1] + ... + 2^248 * h[31].

A curve point (x,y), with coordinates in the range 0 <= x,y < p, is coded as follows. First, encode the y-coordinate as a little-endian string of 32 octets. The most significant bit of the final octet is always zero. To form the encoding of the point, copy the least significant bit of the x-coordinate to the most significant bit of the final octet.

5.1.3. Decoding

Decoding a point, given as a 32-octet string, is a little more complicated.

1. First, interpret the string as an integer in little-endian representation. Bit 255 of this number is the least significant bit of the x-coordinate and denote this value x_0. The y-coordinate is recovered simply by clearing this bit. If the resulting value is >= p, decoding fails.

2. To recover the x-coordinate, the curve equation implies x^2 = (y^2 - 1) / (d y^2 + 1) (mod p). The denominator is always non-zero mod p. Let u = y^2 - 1 and v = d y^2 + 1. To compute the square root of (u/v), the first step is to compute the candidate root x = (u/v)^((p+3)/8). This can be done with the following trick, using a single modular powering for both the inversion of v and the square root:

$$x = (u/v)^{(p+3)/8} = u\, v^3\, (u\, v^7)^{(p-5)/8} \pmod{p}$$

3. Again, there are three cases:

    1. If v x^2 = u (mod p), x is a square root.

    2. If v x^2 = -u (mod p), set x <-- x * 2^((p-1)/4), which is a square root.

    3. Otherwise, no square root exists for modulo p, and decoding fails.

4. Finally, use the x_0 bit to select the right square root. If x = 0, and x_0 = 1, decoding fails. Otherwise, if x_0 != x mod 2, set x <-- p - x. Return the decoded point (x,y).

RFC 8032: Decompression

To recover x-coordinate, let $x^2 = u/v$, and test the potential x given below.

$$u = y^2 - 1 \mod p$$
$$v = dy^2 + 1 \mod p$$
$$x = \left(uv^{-1}\right)^{\frac{p+3}{8}} \equiv uv^3 \left(uv^7\right)^{\frac{p-5}{8}} \mod p$$

OpenSSL Optimization

$$x = u\left(uv\right)^{\frac{p-5}{8}} \mod p$$

OpenSSL Optimization

OpenSSL Optimization: Rationale

Let $x = u\,(uv)^{\frac{p-5}{8}} \mod p$. Want to check if $vx^2 = \pm u \mod p$.

$$\begin{aligned} vx^2 &= u^2 v (uv)^{\frac{p-5}{4}} \\ &= u^2 v (uv)^{\frac{p-1}{4}} (uv)^{-1} \\ &= u\,(uv)^{\frac{p-1}{4}} \end{aligned}$$

Since $x^2 = \frac{u}{v} \mod p$, if such x exists, u,v are both quadratic residues, or both nonresidues, so uv is a quadratic residue. By Euler criterion, $(uv)^{\frac{p-1}{2}} \equiv 1 \mod p$ if valid x, so $(uv)^{\frac{p-1}{4}} \equiv \pm 1 \mod p$.

Field $p = 2^{255} - 19$

Need a fast way to compute modulo $p = 2^{255} - 19$ (pseudo-Mersenne prime).

Computing modulo $p = 2^{255} - 19$

Note that $2^{255} \equiv 19 \mod p$, so $2^{256} \equiv 38 \mod p$.
Given a big endian bignum $B = \underbrace{b_{63} b_{62} \cdots b_{32}}_{b_H} \underbrace{b_{31} \cdots b_0}_{b_L}$, we note that $B = b_H \times 2^{256} + b_L \equiv 38 b_H + b_L = 0\text{b}100110 \times b_H + b_L$.
Then, in computer,
$B \equiv (b_H << 1) + (b_H << 2) + (b_H << 5) + b_L$.